Project overview: German Energy Market Data Pipeline

A simple, point-by-point explanation of what this project does, how it works, and which technologies and skills it uses.

---

1. What is this project about?

- It is an **automated data pipeline** for the **German electricity market**.
- Every hour, Germany produces electricity from the sun (solar) and wind, and electricity is traded at a price per megawatt-hour (€/MWh).
- The project **collects this data automatically**, **cleans and checks it**, **stores it** in a database and in the cloud, and **shows it in a web dashboard**.
- It answers one key question: **How do solar and wind generation affect electricity prices?**

The answer, in short: more solar and wind means much cheaper electricity.

Solar + wind generation	Average price
Lowest 20 % of hours	**241 €/MWh**
Highest 20 % of hours	**58 €/MWh**

- Prices are **lowest around 13:00** (most sun) and **highest around 19:00** (no sun, people at home).
- On sunny, windy weekends prices even became **negative**: there was more electricity than needed.

---

2. Where does the data come from?

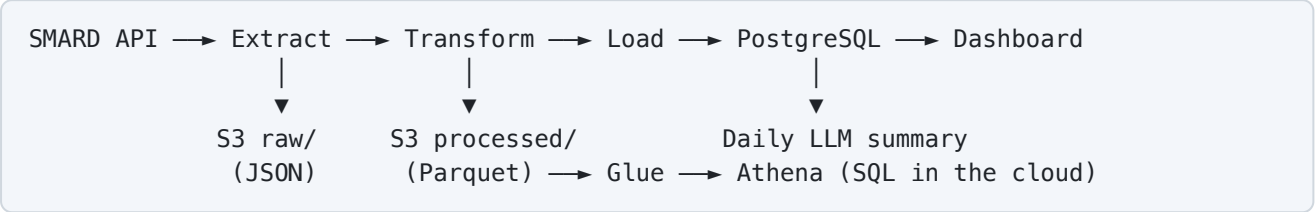
- **SMARD.de**, the electricity market data platform of the **Bundesnetzagentur** (Federal Network Agency). Free, public, no API key.
- Five data series, one value per hour:

Series	Meaning
Day-ahead price	Price of electricity in €/MWh, set one day in advance
Solar	Electricity produced by solar panels
Wind onshore	Electricity from wind turbines on land
Wind offshore	Electricity from wind turbines at sea
Grid load	Total electricity consumption in Germany

- From these, the project calculates the **renewable share**: solar + wind divided by total consumption (49 % on average).

---

3. How does it work? (step by step)



1. **Extract**: downloads the last weeks of data from the SMARD API and saves each answer unchanged as a JSON file ("raw data").
2. **Transform**: combines all series into one table, removes duplicates and incomplete hours, **checks the data quality**, and adds useful columns (renewable share, hour, weekday in German time).
3. **Load**: saves the clean data into a **PostgreSQL** database without ever creating duplicates.
4. **Cloud storage**: uploads raw and cleaned data to **AWS S3**, where it can be queried with SQL using **Athena**.
5. **Summary**: an **LLM** writes a short daily market report from numbers that Python calculated.
6. **Dashboard**: a **Streamlit** website shows key numbers, charts and the summary.
7. **Automation**: one command runs everything; in the cloud, **AWS Lambda** downloads new data **every day automatically**.

---

4. Technologies used

Programming and data processing

Technology	What it is	What it does in this project
Python	Programming language	All pipeline code
pandas	Library for tables of data	Cleaning, combining and calculating
Parquet / PyArrow	Compact file format for tables	Storing the cleaned data efficiently
requests	Library for web requests	Calling the SMARD API with timeouts and retries

Databases and SQL

Technology	What it is	What it does in this project
PostgreSQL	Relational database	Stores all hourly data and the daily summaries
SQL	Database query language	Table design, upserts, analysis queries
psycopg	Python driver for PostgreSQL	Connects the Python code to the database

Cloud (Amazon Web Services)

Technology	What it is	What it does in this project
S3	Cloud file storage ("data lake")	Holds raw JSON and cleaned Parquet files
Glue Data Catalog	Catalog of table definitions	Describes the data in S3 as a table
Athena	SQL queries directly on S3 files	Analysis in the cloud without a database server
Lambda	Runs code without managing a server	Downloads new data every day
EventBridge Scheduler	Cloud timer	Starts the Lambda function daily at 06:00
IAM	Permissions management	Gives each part only the rights it needs
CloudWatch	Logs and monitoring	Stores the Lambda logs
boto3	AWS library for Python	Uploads files to S3

Web app and visualisation

Technology	What it is	What it does in this project
Streamlit	Python framework for web apps	The interactive dashboard
Altair	Chart library	Line, area, bar and scatter charts

LLM (language model)

Technology	What it is	What it does in this project
LLM API (Anthropic Python SDK)	Text generation by a language model	Writes a short, factual daily market summary

Tools and engineering

Technology	What it is	What it does in this project
Docker / Docker Compose	Runs software in containers	Starts PostgreSQL with one command
Git / GitHub	Version control	Tracks every change to the code
pytest	Testing framework	29 automated tests
moto	Simulated AWS	Tests the cloud code without an AWS account
logging	Python's logging module	Clear messages with time and level
python-dotenv	Reads settings from a `.env` file	Keeps passwords and keys out of the code

5. Skills shown in this project

Data engineering

- Building a complete **ETL pipeline** (Extract, Transform, Load).
- Working with a **real public API**: weekly data chunks, delayed and revised values.
- **Data quality checks**: price limits, no negative values, no missing or duplicate hours.
- **Idempotent loading**: running the pipeline again never creates duplicates.
- Storing data in a **data lake** with **partitioning by date**.

SQL and databases

- Designing a table with **primary key**, exact number types and **CHECK constraints**.
- **Upserts** (`INSERT … ON CONFLICT DO UPDATE`).
- Analysis with **GROUP BY**, **window functions** (`ntile`) and **correlation**.
- Cloud SQL with **Athena** and **partition projection**.

Python

- Clean, commented, modular code (one file per pipeline step).
- pandas for cleaning, merging, reshaping and aggregating data.
- Error handling, retries with **exponential backoff**, and timeouts.

Web apps and data visualisation

- An interactive **Streamlit** dashboard with a date filter, KPIs and four charts.
- Good chart design: no double y-axis, consistent colours, tooltips.
- **Caching**, so the database is not queried on every click.

Cloud (AWS)

- **Serverless** automation with Lambda and EventBridge Scheduler.
- **Least-privilege IAM**: separate roles with minimal permissions.
- **Cost awareness**: no Glue crawler, a query scan limit, log retention, a budget alert.

LLM-based automation

- Generating text reports with an **LLM API**.
- **Grounding**: the model only receives numbers calculated by Python, so it cannot invent statistics.
- Storing results once per day instead of calling the model on every page view.

Software engineering practices

- **Automated tests** with pytest (29 tests), including edge cases.
- **Mocking**: fake API, fake database client and simulated AWS in tests.
- **Logging** instead of print, and **exit codes** for automation.
- **Docker** for a reproducible database.
- **Git** with clear commit messages.
- **Security**: no passwords or keys in the code (`.env` and `~/.aws`, both outside git).

---

6. Important concepts, explained simply

Concept	Simple explanation
ETL	Extract (download) → Transform (clean) → Load (save). The basic pattern of data engineering.
Raw data	The original, unchanged data. Kept so the later steps can always be repeated.
Data validation	Automatic checks that stop the pipeline when the data looks wrong.
Idempotent	Doing something twice gives the same result as doing it once.
Upsert	"Insert or update": new rows are added, existing rows are updated.
Primary key	A column whose value must be unique, here the hour.
UTC	World time without summer/winter time changes. Used for storage, so no hour is ever duplicated or skipped.
Daylight saving time	On 25 Oct 2026, 02:00 happened twice in Germany. The pipeline and tests handle this.
Data lake	Cheap cloud storage for files of any kind (here S3).
Parquet	A compressed, column-based file format, much smaller and faster than CSV.
Partitioning	Splitting data into folders by day, so queries read only the days they need.
Serverless	Code runs in the cloud only when needed; no server to manage.
Least privilege	Every user or program gets only the permissions it really needs.
Caching	Keeping results in memory for a while instead of fetching them again.
Unit test	A small automatic check that one function works correctly.
Mocking	Replacing a real service (API, cloud) with a fake one in tests.
Exit code	A number a program returns when it ends: 0 = success, 1 = failure.

---

7. How to run it (short version)

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt
cp .env.example .env
docker compose up -d
python src/pipeline.py
streamlit run dashboard/app.py
pytest
```

After `cp .env.example .env`, set your own database password in `.env`. `docker compose up -d` starts the database, `python src/pipeline.py` collects, cleans and loads the data, `streamlit run dashboard/app.py` opens the dashboard at http://localhost:8501, and `pytest` runs the tests.

- The AWS part is optional: see docs/AWS_SETUP.md.
- The daily summary is optional: it needs an API key in `.env`.

---

8. Project structure

Folder / file	Content
`src/extract.py`	Downloads data from SMARD
`src/transform.py`	Cleans, checks and enriches the data
`src/load.py`	Saves data into PostgreSQL
`src/s3_upload.py`	Uploads data to AWS S3
`src/summary.py`	Creates the daily LLM summary
`src/pipeline.py`	Runs all steps with one command
`dashboard/app.py`	The Streamlit dashboard
`aws/`	Lambda function, build script, IAM permission files
`sql/`	Database schema, analysis queries, Athena queries
`tests/`	Automated tests
`docs/`	Documentation (this file and the AWS guide)
`docker-compose.yml`	Starts the PostgreSQL database

---

9. Possible next steps

- Add more data, e.g. other energy sources (gas, coal) or neighbouring countries.
- Predict tomorrow's price with a simple machine-learning model.

- Host the dashboard online (e.g. Streamlit Community Cloud).
- Run the whole pipeline in the cloud, not only the extract step.
- Add automatic tests on GitHub (GitHub Actions) for every push.